

Create themes

Create a local theme when CSS overrides are not enough: for example, when you need to change HTML templates, add JavaScript, replace shared partials, or customize the Typst notebook template.

Eject a built-in theme

Built-in themes are compiled into the *Calepin* binary, so you cannot edit them directly. Instead, copy one into your project and edit the copy:

```
calepin new theme          # copies calepin to themes/calepin/
calepin new theme --theme academic # copies academic to themes/academic/
calepin new theme themes/my-theme --theme academic
```

Then point your site or compile command at the copy:

```
theme = "themes/calepin"
```

The copy is yours: edit its HTML, CSS, JavaScript, `theme.toml`, and notebook template files freely, and check them into version control. *Calepin* upgrades will never touch them. Ejected shared files are written beside the theme in `themes/shared/`.

Start small

A local theme can be small. Missing standard entry files fall back to the built-in `calepin` theme, so a local theme can override just `layouts/webpage.html`, just `layouts/notebook.html`, or just `notebook.typ.jinja`. Supporting files such as partials, styles, and scripts come from the selected theme plus any imports declared in `theme.toml`.

HTML templates

For notebooks compiled as single HTML documents, the relevant layout is `layouts/notebook.html`. For websites, most pages use `layouts/webpage.html`.

Templates can access:

- `doc.head`
- `doc.body_open`
- `doc.body`
- `doc.body_close`
- `doc.title`
- `site.sidebar`
- `site.sidebar_sections`
- `site.menus`
- `site.menu_list`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`

- `site.favicon`
- `site.current_url`
- `site.page_title`
- `styles`
- `scripts`
- `syntax_css`
- `theme`
- `target`

Navigation entries expose `href`, `label`, `label_html`, and `active`. `site.menus` is a map from menu name to navigation entries, such as `site.menus.main` and `site.menus.social`. `site.menu_list` contains the same menus as `{ name, items }` records for themes that need to iterate over every configured menu.

Here is a minimal `layouts/notebook.html` for single-file HTML output:

```
{{ doc.head }}
<title>{{ doc.title }}</title>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.min.css">
{% for style in styles %}
<style>
{{ style.css }}
</style>
{% endfor %}
{{ doc.body_open }}
<header class="document-header">
  <a href="index.html">Home</a>
  <button type="button" data-calepin-theme-toggle>Theme</button>
</header>
<main class="container">
  {{ doc.body }}
</main>
{% for script in scripts %}
<script>
{{ script.content }}
</script>
{% endfor %}
{{ doc.body_close }}
```

`doc.head`, `doc.body_open`, and `doc.body_close` come from Typst's generated HTML. Keep them in the template unless you are deliberately replacing the whole document shell.

Website layouts

A website page can select a different HTML layout from the active theme with `layout` in its `<website-metadata>`:

```
#metadata((
  title: "Landing page",
  layout: "layouts/landing.html",
)) <website-metadata>
```

The `layout` value is an explicit path inside the active theme. *Calepin* uses it exactly as written: it does not add `layouts/`, does not add `.html`, and does not fall back to `layouts/webpage.html` if the file is missing. The path must name a relative `.html` file that stays inside the theme directory.

Partials

A partial is a reusable MiniJinja template fragment stored under `partials/`. Use partials for repeated HTML such as a header, footer, navigation list, search box, or analytics snippet.

Include a partial from `layouts/webpage.html`, `layouts/notebook.html`, a page-specific layout, or another partial:

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them.

Shared files

Themes can opt into shared partials, CSS, and JavaScript with `theme.toml`. These are the pieces that the built-in themes use for metadata, stylesheet/script wiring, typography, syntax highlighting, code output, dark mode, language selection, theme toggles, search, and copy buttons.

```
[shared]
partials = ["site-meta.html", "theme-init.html", "styles.html", "scripts.html", "pagefind-modal.html",
"theme-toggle.html"]
styles = ["theme.css", "code.css", "widgets.css"]
scripts = ["theme-toggle.js", "language-picker.js", "copy-code.js"]
```

Shared imports load in the order listed in `theme.toml`. Files in the theme's own `partials/`, `styles/`, and `scripts/` directories load after the shared imports in filename order. If a theme-local file has the same filename as a shared import, the local file overrides that import.

Import names are filenames, not paths: use `theme.css`, not `styles/theme.css` or `../theme.css`. For local directory themes, *Calepin* first checks the theme's own file, then `themes/shared/`, then the embedded shared library shipped with *Calepin*.

Notebook Typst templates

Notebook outputs use a Typst-source MiniJinja template named `notebook.typ.jinja`:

```
themes/my-theme/
notebook.typ.jinja
```

Typst already has a complete styling system based on `#set` and `#show` rules. *Calepin*'s notebook theme layer adds one optional step before Typst runs: `notebook.typ.jinja` is rendered with MiniJinja and must produce Typst source.

Every bundled theme includes `notebook.typ.jinja`. To customize it, eject a bundle:

```
calepin new theme
```

Then edit `themes/calepin/notebook.typ.jinja` and select that local theme:

```
calepin compile paper.typ --theme themes/calepin
```

Use `document.body` where the notebook source should appear:

```
#set page(
  paper: "us-letter",
  margin: (x: 1in, y: 0.85in),
  numbering: "1",
)

#set text(font: "Libertinus Serif", size: 10.5pt)

{{ document.body }}
```

`notebook.typ.jinja` receives:

- `theme`: the local theme directory name

- `target`: notebook
- `document.path`: the root-relative `.typ` input path
- `document.dir`: the root-relative input directory
- `document.stem`: the input filename without `.typ`
- `document.body`: the staged notebook source as a Typst `#include`
- `document.meta`: metadata from `#metadata(...)` <website-metadata>
- `params`: document parameters after CLI overrides

If `notebook.typ.jinja` does not reference `document.body`, *Calepin* treats it like a prelude and includes the notebook source after the rendered template.

For output-specific branches, use Typst's runtime input instead of MiniJinja:

```
#let is-html = sys.inputs.at("calepin-target", default: "paged") == "html"
```

Set `theme = false` or use an empty `notebook.typ.jinja` to disable notebook Typst styling. Local themes that still use the older `paged.typ.jinja` filename continue to work, but new themes should use `notebook.typ.jinja`.